Autores: John Sebastian Ceballos Beltran – Edwar Alexander Benito Basante

Paso 1: Construcción de la imagen (Reto 1)

Justificación:

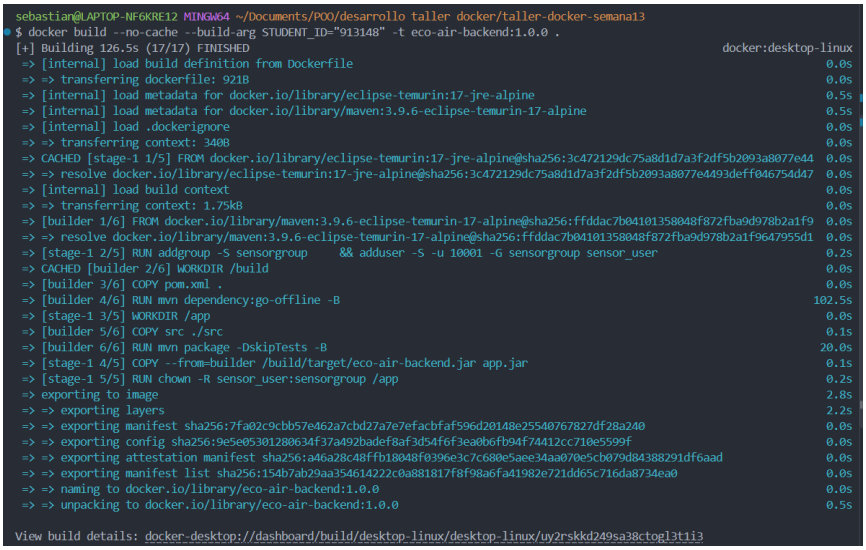
Se construyó la imagen con el comando docker build --build-arg STUDENT_ID="913148" -t eco-air-backend:1.0.0 .. Se usó --no-cache para evidenciar el proceso completo. El build terminó sin errores en 126.5 s.

El Dockerfile tiene dos etapas. En la etapa builder se compila el proyecto con Maven, y en la etapa runtime solo se copia el .jar resultante. Por eso la imagen final no contiene Maven ni el código fuente, lo que la hace más ligera y reduce la superficie de ataque.

Primero se copia solo el pom.xml y se ejecuta mvn dependency:go-offline -B, que tardó 102.5 s. Después se copia src/ y se empaqueta con mvn package -DskipTests -B, que tardó 20 s. Se separaron así para que Docker guarde las dependencias en una capa propia. Mientras el pom.xml no cambie, un cambio en el código solo repite la compilación (la capa de 20 s) y no la descarga (la de 102.5 s).

El argumento STUDENT_ID se recibe en tiempo de construcción y se convierte en la variable de entorno OPERATOR_ID, que identifica la imagen con el código del estudiante.

En la etapa runtime se crea el grupo sensorgroup y el usuario sensor_user (UID 10001), se entrega la titularidad de /app a ese usuario y se cambia a él con USER antes del ENTRYPOINT. Así la aplicación nunca corre como root. También se definió JAVA_OPTS para que la JVM respete el límite de memoria del contenedor, con MaxRAMPercentage=75.0, en vez de intentar usar toda la RAM del servidor.



Paso 2: desacoplamiento de capas (Reto 2.1)

Justificación**:**

Se ejecutó docker history sobre la imagen eco-air-backend:1.0.0 para verificar qué contiene cada capa.

En el historial solo aparecen capas de la etapa de ejecución: la base Alpine (alpine-minirootfs, 9.08MB), el JRE de Java 17 (44MB y 141MB), y las capas que agregamos: creación del usuario (addgroup, 41kB), WORKDIR /app, la copia del .jar (49MB) y el chown (49MB). No aparece ninguna capa de Maven, ni mvn dependency:go-offline, ni mvn package, ni el COPY src. Esto demuestra que la etapa builder se descartó y que solo el .jar pasó a la imagen final.

También se observa en las capas la configuración de seguridad y de ejecución: USER sensor_user (el proceso no corre como root), ENV OPERATOR_ID=913148 (el código de estudiante inyectado en el build), ENV JAVA_OPTS (parámetros de la JVM), EXPOSE 8080 y el ENTRYPOINT que lanza java.

Existen dos observaciones que se pueden hacer al revisar los logs de la terminal: COPY ... eco-air-backend.jar y RUN chown -R ... pesan 49MB cada una. El chown reescribe el archivo en una capa nueva, por lo que el .jar ocupa espacio dos veces en el historial. Se aceptó porque el taller exige cambiar la titularidad de /app. Una alternativa sería usar COPY --chown=sensor_user:sensorgroup, pero en este caso se siguió la instrucción del taller tal cual.

Las capas debajo de ARG STUDENT_ID (JRE, Alpine) pertenecen a la imagen base eclipse-temurin:17-jre-alpine, no a nuestro código.

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ docker history --human --format "table {{.Size}}\t{{.CreatedBy}}" eco-air-backend:1.0.0
SIZE      CREATED BY
0B        ENTRYPOINT ["sh" "-c" "exec java $JAVA_OPTS …
0B        USER sensor_user
0B        EXPOSE [8080/tcp]
0B        ENV OPERATOR_ID=913148
0B        ENV JAVA_OPTS=-XX:+UseContainerSupport -XX:M…
49MB      RUN |1 STUDENT_ID=913148 /bin/sh -c chown -R…
49MB      COPY /build/target/eco-air-backend.jar app.j…
8.19kB    WORKDIR /app
41kB      RUN |1 STUDENT_ID=913148 /bin/sh -c addgroup…
0B        ARG STUDENT_ID=913148
0B        ENTRYPOINT ["/__cacert_entrypoint.sh"]
12.3kB    COPY --chmod=755 entrypoint.sh /__cacert_ent…
12.3kB    RUN /bin/sh -c set -eux;     echo "Verifying…
141MB     RUN /bin/sh -c set -eux;     ARCH="$(apk --p…
0B        ENV JAVA_VERSION=jdk-17.0.20.1+1
44MB      RUN /bin/sh -c set -eux;     apk add --no-ca…
0B        ENV LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_AL…
0B        ENV PATH=/opt/java/openjdk/bin:/usr/local/sb…
0B        ENV JAVA_HOME=/opt/java/openjdk
0B        CMD ["/bin/sh"]
9.08MB    ADD alpine-minirootfs-3.24.2-x86_64.tar.gz /…
```

Paso 3: aislamiento de privilegios (Reto 2.2)

Justificación:

En el momento en el que se inicia el contenedor en modo detached (-d) con el nombre eco_air_active y el puerto 8080, se le asigns un límite de memoria de 192 MB (-m 192m) con --memory-swap 192m, que es el umbral mínimo estable calibrado en el Reto 3.2. Así el mismo contenedor sirve para todas las pruebas posteriores.

Con docker top eco_air_active -o pid,user,comm se consultó la tabla de procesos del contenedor, vista desde el host. La salida muestra un único proceso, java (PID 426), cuyo usuario es 10001. Ese UID es el que se le asignó a sensor_user en el Dockerfile, no el 0 de root. El host muestra el número y no el nombre porque no conoce el archivo /etc/passwd de dentro del contenedor. Por eso se confirma el nombre en el paso siguiente.

Esto demuestra que la aplicación no corre con privilegios de root. Si un atacante lograra ejecutar código a través de ella, tendría los permisos limitados de sensor_user, y no los de administrador.

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ docker rm -f eco_air_active

sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ docker run -d --name eco_air_active -p 8080:8080 -m 192m --memory-swap 192m eco-air-backend:1.0.0
2f77f14114c34640beb3fac9efe1198d43ebdd70e68f0b1ca609cc096b5b60d0

sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ docker top eco_air_active -o pid,user,comm
PID                 USER                COMMAND
426                 10001               java
```

Paso 3: aislamiento de privilegios (Reto 2.2)

Justificación:

Se inició el contenedor en modo detached con docker run -d --name eco_air_active -p 8080:8080 -m 256m --memory-swap 256m eco-air-backend:1.0.0. Se le asignó un límite de 256 MB porque con 192 MB el contenedor fue eliminado por falta de memoria en este equipo (ver Reto 3.2). La aplicación arrancó correctamente en 5.64 segundos.

Con docker top eco_air_active -o pid,user,comm se consultó la tabla de procesos del contenedor desde el host. La salida muestra un único proceso, java (PID 630), cuyo usuario es 10001, y no root. El host muestra el número porque no conoce el /etc/passwd del contenedor, donde está definido el nombre.

Para relacionar el UID con el nombre se ejecutó id sensor_user dentro del contenedor, que devolvió uid=10001(sensor_user) gid=101(sensorgroup). Además, ps -o pid,user,comm muestra java como PID 1 bajo el usuario sensor_u (el nombre de sensor_user aparece cortado por el ancho de la columna de ps en Alpine).

Esto demuestra que el comando java no se ejecuta como root, sino bajo sensor_user. Si un atacante lograra ejecutar código a través de la aplicación, tendría solo los permisos limitados de ese usuario.

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ docker top eco_air_active -o pid,user,comm
PID                 USER                COMMAND
630                 10001               java
```

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ winpty docker exec -t eco_air_active id sensor_user
uid=10001(sensor_user) gid=101(sensorgroup) groups=101(sensorgroup),101(sensorgroup)
failed to resize tty, using default size

sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ winpty docker exec -t eco_air_active ps -o pid,user,comm
PID   USER     COMMAND
    1 sensor_u java
   52 sensor_u ps
```

Paso 5: inmutabilidad de runtime (Reto 2.3)

Se simuló un ataque que intenta dejar un archivo malicioso en un directorio del sistema, ejecutando touch //etc/crontab_fake dentro del contenedor. (La doble barra es equivalente a /etc/ y se usó solo para evitar que Git Bash en Windows convirtiera la ruta.)

El resultado fue touch: cannot touch '//etc/crontab_fake': Permission denied. La escritura fue rechazada porque el proceso corre como sensor_user (UID 10001), y el directorio /etc pertenece a root, sin permiso de escritura para otros usuarios.

Esto demuestra que, aunque un atacante comprometiera la aplicación, no podría modificar archivos del sistema ni crear tareas programadas (cron) para persistir en el contenedor. Es una consecuencia directa de haber creado el usuario sin privilegios y de cambiar a él con USER antes del ENTRYPOINT.

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ winpty docker exec -t eco_air_active touch //etc/crontab_fake
touch: cannot touch '//etc/crontab_fake': Permission denied
```

Pasos 6 y 7: auditoría de artefactos de desarrollo (Reto 2.4)

Se verificó que la imagen de producción no contenga herramientas de compilación ni código fuente.

Con docker exec eco_air_active which mvn no se obtuvo ninguna salida y el código de salida fue 1, es decir, Maven no existe en el contenedor. Con find / -name "*.java" no se listó ningún archivo .java. Las dos líneas que aparecen (/root y /proc/tty/driver con Permission denied) son directorios que find no puede leer porque el proceso corre como sensor_user y no como root. Ese es también el motivo por el que el código de salida es 1: find lo devuelve cuando no pudo leer algún directorio, no porque haya encontrado algo. Ningún resultado de la búsqueda es un archivo .java.

Esto demuestra que el build multietapa funciona: Maven y el código fuente quedaron en la etapa builder, que se descartó, y a la imagen final solo pasó el .jar. Una imagen sin compiladores ni fuentes es más pequeña y reduce la superficie de ataque, porque un intruso no tiene herramientas para compilar ni el código para estudiar vulnerabilidades. Además, las dos líneas de Permission denied refuerzan lo visto en los pasos anteriores: el proceso no tiene privilegios de root.

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ winpty docker exec -t eco_air_active which mvn

sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ echo $?
1
```

```
sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ winpty docker exec -t eco_air_active sh -c 'find / -name "*.java" 2>/dev/null'

sebastian@LAPTOP-NF6KRE12 MINGW64 ~/Documents/POO/desarrollo taller docker/taller-docker-semana13
$ echo $?
1
```

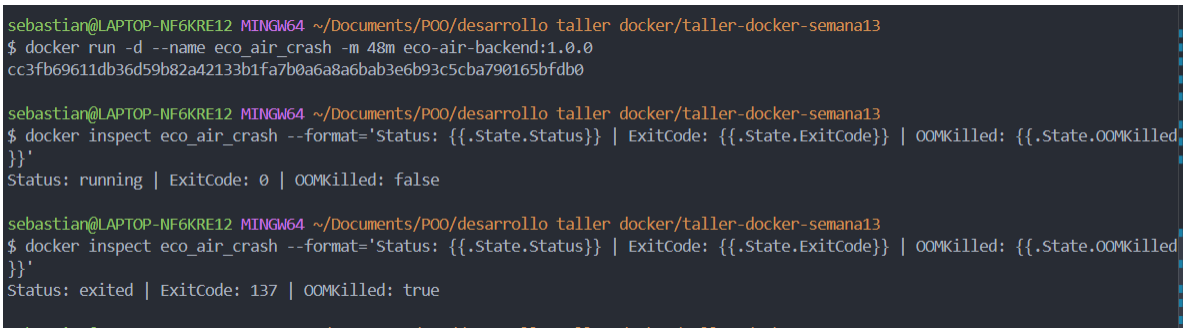
Paso 8: colapso por memoria (Reto 3.1)

Se ejecutó una instancia con un límite de memoria de solo 48 MB (docker run -d --name eco_air_crash -m 48m ...), insuficiente para levantar Spring Boot con Hibernate. Una primera consulta con docker inspect, hecha inmediatamente, mostró Status: running porque el contenedor aún estaba arrancando. Al repetirla unos segundos después, la salida fue:

Status: exited | ExitCode: 137 | OOMKilled: false → true (es decir: Status: exited | ExitCode: 137 | OOMKilled: true).

Esto significa que el contenedor ya no está corriendo (exited) y que su proceso terminó con el código 137, que equivale a 128 + 9: la señal SIGKILL, que el sistema envía para matar un proceso a la fuerza. OOMKilled: true confirma la causa: el kernel de Linux, a través del grupo de control (cgroup) de memoria, mató el proceso porque intentó usar más de los 48 MB asignados (OOM = Out Of Memory).

El resultado demuestra que el límite de memoria es efectivo y protege al servidor: aunque la JVM necesite más RAM de la permitida, el contenedor falla de forma aislada y no afecta al resto del sistema. También muestra que 48 MB no es un valor viable para esta aplicación, lo que justifica calibrar un umbral mínimo operativo en el siguiente reto.



Reto 3: calibración de cgroups y captura del OOM

3.1 Inducción del colapso por memoria (48 MB)

Se ejecutó una instancia con un límite de 48 MB (docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0), insuficiente para levantar Spring Boot con Hibernate. Una primera consulta con docker inspect, hecha de inmediato, mostró Status: running porque el contenedor aún estaba arrancando. Unos segundos después, la salida fue:

Status: exited | ExitCode: 137 | OOMKilled: true

El contenedor ya no corre (exited) y su proceso terminó con el código 137, que equivale a 128 + 9: la señal SIGKILL. OOMKilled: true confirma la causa: el kernel, a través del cgroup de memoria, mató el proceso porque intentó usar más de los 48 MB asignados (Out Of Memory). Esto demuestra que el límite es efectivo: el fallo queda aislado en el contenedor y no afecta al servidor.

3.2 Calibración del límite operativo mínimo

Se probaron límites crecientes, siempre con --memory-swap igual a -m para que el swap no enmascarara el límite real de RAM.

Límite -m	Resultado
192m	exited, ExitCode: 137, OOMKilled: true
208m	Arrancó en 5.638 s, estable tras 30 peticiones, 85.72 % de memoria
224m	Arrancó en 5.466 s, estable tras 30 peticiones, 78.61 % de memoria

Con 192 MB el contenedor fue terminado por el kernel durante la carga de Hibernate, sin llegar a mostrar Started EcoAirApplication. [Opcional, solo si tu compañero tiene su propia evidencia: El mismo comportamiento se observó en el equipo de un compañero.] Esto evidencia que el umbral depende de los recursos del host. En otras pruebas ese mismo valor llegó a arrancar con muy poco margen.

Con 208 MB el contenedor cumplió todos los criterios: arrancó en menos de 15 segundos y se mantuvo estable. Tras 30 peticiones a la API, docker stats mostró:

CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM % NET I/O BLOCK I/O PIDS

fdbe0289e979 eco_air_active 0.31% 178.3MiB / 208MiB 85.72% 18.5kB / 14.6kB 88.3MB / 73.7kB 31

Se concluye que el umbral mínimo operativo es 208 MB. Como el consumo llega al 85.72 % del límite, en un entorno real convendría dejar margen, por ejemplo 224 MB, que usó el 78.61 %.

